

Informe de Laboratorios — Sesión 6

Migraciones y Seeds en el Pipeline; Gestión de Secretos

Estudiante: Marianela Saravia Ortega

Módulo: 4 — Integración y Despliegue Continuo

Proyecto: Gestor de Tareas (React + Express + PostgreSQL)

Repositorio: github.com/nel39448-art/mod4-task-manager

Revisor cruzado: Cypher-dev-bo

Fecha: 17 de julio de 2026

Introducción

En esta sesión hice dos cosas en el pipeline: que aplique el esquema de la base de datos por su cuenta (migraciones y seed contra un PostgreSQL de prueba) y que ninguna credencial quede escrita en texto plano en el repositorio.

El proyecto ya traía PostgreSQL y Prisma de la Sesión 5, con las migraciones versionadas en git, así que partí de ahí. Como la app no tiene autenticación, la única variable sensible es `DATABASE_URL` (lleva usuario y contraseña); no hay un `JWT_SECRET` como supone la guía, así que trabajé solo con `DATABASE_URL`.

Laboratorio 1 — Variables en GitHub Secrets

Guardé `DATABASE_URL` como secreto del repositorio en *Settings* → *Secrets and variables* → *Actions*. El pipeline lo lee con `${{ secrets.DATABASE_URL }}` y agregué un paso que confirma que el secreto está disponible sin imprimir su valor.

```
# .github/workflows/ci.yml (paso del job de migraciones)
- name: Verificar secreto DATABASE_URL (sin exponerlo)
  env:
    SECRET_DB_URL: ${{ secrets.DATABASE_URL }}
  run: |
    if [ -n "$SECRET_DB_URL" ]; then
      echo "Secret DATABASE_URL presente y enmascarado por GitHub."
    fi
```

En el log de Actions se ve que GitHub reemplaza el valor por `***`. El secreto nunca aparece en texto plano, ni siquiera parcialmente.

```
env:
  DATABASE_URL: ***localhost:5432/testdb
  SECRET_DB_URL: ***
Secret DATABASE_URL presente y enmascarado por GitHub.
```

No inyecté el secreto al job de pruebas. Esos tests corren en memoria a propósito (sin `DATABASE_URL`), y ponérselo haría que intentaran conectarse a una base que ahí no existe y fallaran. Por eso el secreto se usa en el job de migraciones, no en el de pruebas.

Laboratorio 2 — Migraciones en el pipeline

Agregué un job `Migraciones y Seeds` al `ci.yml`. Levanta un PostgreSQL temporal como servicio de GitHub Actions (existe solo durante esa ejecución), espera a que pase el healthcheck, y aplica las migraciones con `prisma migrate deploy`.

```
migraciones:
  name: Migraciones y Seeds
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:16-alpine
      env: { POSTGRES_USER: testuser, POSTGRES_PASSWORD: testpass, POSTGRES_DB: testdb }
      ports: [ 5432:5432 ]
      options: >-
        --health-cmd pg_isready --health-interval 5s
        --health-timeout 5s --health-retries 5
  env:
    DATABASE_URL: postgresql://testuser:testpass@localhost:5432/testdb
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: '20', cache: 'npm' }
    - run: npm ci
    - run: npx prisma migrate deploy
    - run: npx prisma db seed
```

Este PostgreSQL de prueba usa credenciales fijas (`testuser/testpass`) y no un secreto, porque es descartable y no es accesible desde fuera. La base real de producción sí usa el secreto, como en el Laboratorio 1.

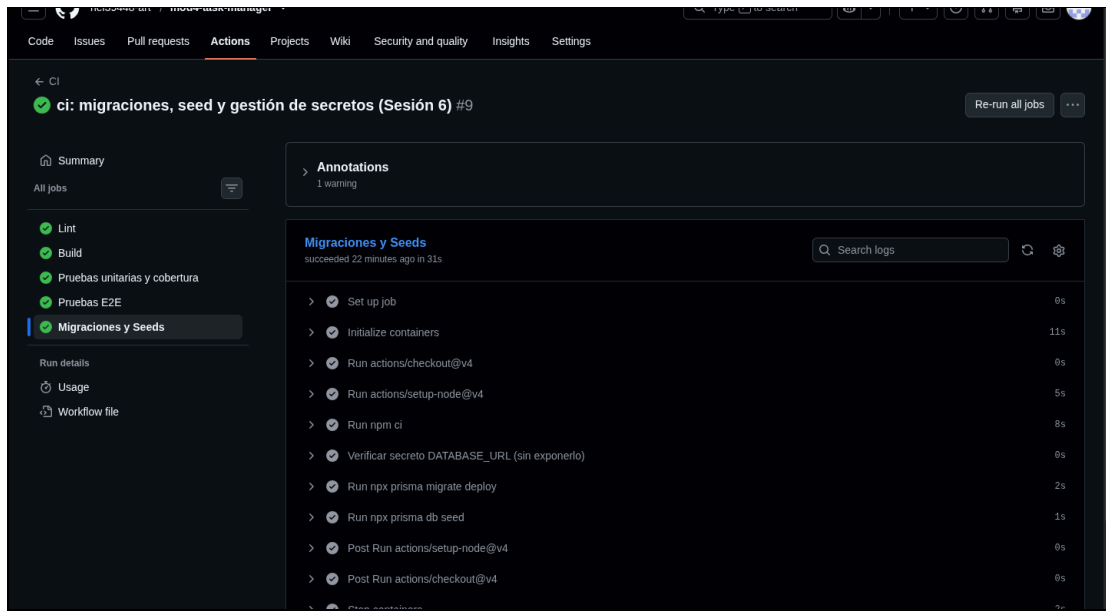


Figura 1. El job "Migraciones y Seeds" en verde, con los pasos de verificación del secreto, `prisma migrate deploy` y `prisma db seed`. Es el 5.º check obligatorio de `main`.

Laboratorio 3 — Seed reproducible

Escribí `prisma/seed.js` con `upsert`, que actualiza si el registro ya existe en vez de crear uno nuevo, y lo registré en `package.json` como `prisma.seed`. Lo escribí con `import` porque el proyecto usa módulos ESM.

```
// prisma/seed.js
import { PrismaClient } from '@prisma/client'

const prisma = new PrismaClient()

async function main() {
  await prisma.tarea.upsert({
    where: { id: 1 },
    update: {},
    create: { id: 1, titulo: 'Tarea de ejemplo para pruebas', completada: false },
  })
}

main().then(() => prisma.$disconnect()).catch(async (e) => {
  console.error(e); await prisma.$disconnect(); process.exit(1)
})
```

Lo corrí dos veces seguidas contra una base de prueba. La segunda corrida no falla ni duplica: la tabla queda con una sola fila.

```
$ npx prisma db seed # 1.ª vez
🌱 The seed command has been executed.
$ npx prisma db seed # 2.ª vez
🌱 The seed command has been executed.
$ psql -c 'SELECT count(*) FROM "Tarea";'
1 (una sola fila, sin duplicar)
```

Laboratorio 3 — Auditoría de Fugas

Este ejercicio se hace en una rama de práctica que después se borra, sin fusionar a `main`. Lo hice con valores obviamente falsos y en una rama local, para no subir ni un secreto de práctica a GitHub.

Plantar las fugas

Puse dos fugas ficticias. La primera, una clave escrita directamente en un archivo. La segunda, un archivo `.env.filtrado` que forcé a subir saltando el `.gitignore`, y que después borré en otro commit (sin limpiar el historial, para ver que eso no basta).

Encontrarlas

La fuga del código actual se encuentra con `grep`. La del historial no aparece en el código actual, pero `git log -p` la muestra igual, porque el commit que la agregó sigue en el historial aunque el archivo ya se haya borrado.

```
$ grep -rn "apiKey\\|sk_test\\|SECRET" --include="*.js" .
config-demo.js:3:const apiKey = 'sk_test_FALS0123456789' // TODO: mover a variable de entorno

$ git log -p --all -- .env.filtrado
commit 7f64df5 (chore: eliminar archivo de configuracion)
-DATABASE_URL=postgresql://usuario:contrasenaDePrueba999@host:5432/db
commit 2df45ad (chore: archivo de configuracion - fuga intencional)
+DATABASE_URL=postgresql://usuario:contrasenaDePrueba999@host:5432/db
```

Remediarlas

La primera fuga se corrige reemplazando el valor por una lectura de variable de entorno (`const apiKey = process.env.API_KEY`) y guardando la clave como secreto. Después del cambio, `grep` ya no encuentra nada.

```
$ grep -rn "sk_test" --include="*.js" .
(sin coincidencias: la clave ya no está en el código)
```

Con la segunda fuga, borrar el archivo no alcanza: sigue siendo recuperable en el historial. Si hubiera sido una credencial real, lo correcto sería rotarla (generar una nueva e invalidar la anterior) y limpiar el historial con una herramienta como `git filter-repo` o BFG. Al terminar borré la rama de práctica, que nunca se fusiona a `main`.

Cierre

Todo el trabajo permanente (secreto, job de migraciones y seed) entró a `main` por un Pull Request con los cinco checks en verde y la aprobación del revisor cruzado Cypher-dev-bo. El nuevo job quedó como quinto check obligatorio de la rama.

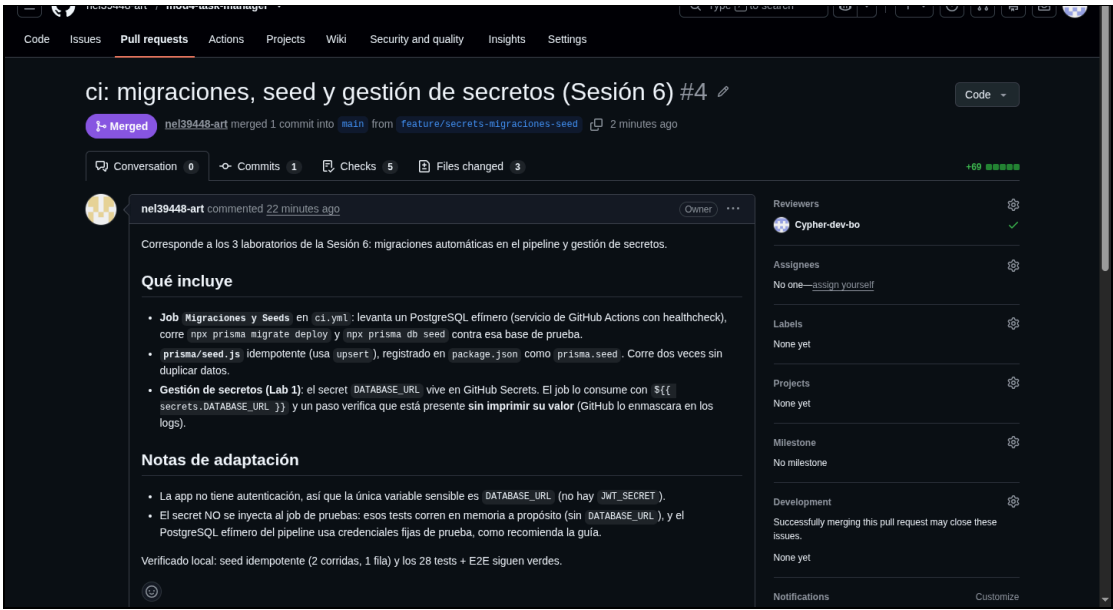


Figura 2. PR #4 "migraciones, seed y gestión de secretos", mergeado a `main`, aprobado por Cypher-dev-bo y con los 5 checks en verde.

Estado del entregable

Requisito	Estado
Variables sensibles en GitHub Secrets (aquí <code>DATABASE_URL</code>), ninguna en texto plano	Cumplido (Lab 1)
El pipeline aplica <code>prisma migrate deploy</code> contra una base de prueba efímera	Cumplido (Figura 1)
El seed corre reproducible, sin duplicar datos	Cumplido (2 corridas, 1 fila)
Auditoría de Fugas: encontrar y remediar ambas fugas ficticias	Cumplido
La rama <code>feature/auditoria-fugas</code> fue eliminada al terminar	Cumplido

Insignias obtenidas

- **Auditor de Fugas:** encontré las dos fugas (la del código con `grep` y la del historial con `git log -p`).
- **Guardián de Secretos:** remedié la fuga del código y moví la variable sensible a GitHub Secrets.
- **Sembrador Reproducible:** el seed corre dos veces sin duplicar datos.